

파이토치 딥러닝 프로그래밍

객체검출 2_One-Stage Detector

▶ One-Stage Detector와 YOLO

- One-stage Detector 의 핵심 장점
- YOLO (You Only Look Once)
- YOLO 출력 텐서의 이해
- Feature Pyramid Network (FPN)과 Darknet-53
- YOLO 손실함수 : 3가지 손실의 조합
- 평가지표: Precision, Recall, mAP

Two-Stage Detector

대표 모델: Faster R-CNN

Region Proposal과 Classification을 순차적으로 수행하는 방식입니다. 먼저 RPN(Region Proposal Network)이 객체가 있을 만한 영역을 제안하고, 이후 각 영역에 대해 분류와 위치 정제를 진행합니다.

- 높은 정확도 (mAP 우선)
- 느린 추론 속도 (~10 FPS)
- 복잡한 파이프라인
- 학습이 단계별로 진행

One-Stage Detector

대표 모델: YOLO, SSD

Region Proposal과 Classification을 동시에 수행하는 통합 방식입니다. 이미지를 그리드로 나누고 각 그리드에서 직접 객체를 예측하여, 단일 네트워크 통과만으로 검출을 완료합니다.

- 빠른 속도 (45+ FPS, 실시간 가능)
- 단순한 구조 (End-to-end)
- 전역 정보 활용
- 배경 오류 감소



실시간 처리

45+ FPS (YOLO v2 기준)의 빠른 속도로 실시간 응용에 적합합니다. 자율주행, 보안 감시 등 즉각적인 반응이 필요한 시스템에서 활용됩니다.



단순한 구조

복잡한 파이프라인 없이 End-to-end 학습이 가능합니다. 단일 네트워크로 모든 작업을 처리하여 구현과 배포가 용이합니다.



전역 정보 활용

이미지 전체를 한 번에 처리하여 객체 간의 맥락적 관계를 파악합니다. 지역적 패치만 보는 것이 아니라 전체 장면을 이해합니다.



배경 오류 감소

전체 맥락을 고려하여 배경을 객체로 오인하는 False Positive가 줄어듭니다. Region Proposal 방식의 치명적 단점을 극복합니다.



- 이미지를 단 한 번만 봐도 모든 객체를 동시에 검출할 수 있다는 혁신적 아이디어

01

그리드 분할

입력 이미지를 $S \times S$ 그리드로 균등 분할합니다. YOLO v1은 7×7 , v2는 13×13 그리드를 사용합니다.

02

박스 예측

각 그리드 셀이 B개의 bounding box를 예측합니다. 각 박스는 (x, y, w, h, confidence)를 포함합니다.

03

클래스 확률

각 그리드 셀이 클래스 확률을 예측합니다. C개 클래스에 대한 조건부 확률 분포를 출력합니다.

04

동시 예측

하나의 CNN으로 모든 박스와 클래스를 동시에 예측합니다. 단일 forward pass로 전체 검출이 완료됩니다.

YOLO v1

출력: $7 \times 7 \times 30$

- $S = 7$ (그리드 크기)
- $B = 2$ (박스 개수)
- $C = 20$ (PASCAL VOC 클래스)
- 총 98개 예측 박스

각 셀: 2개 박스 $\times$ 5 + 20개 클래스 확률 = 30차원 벡터

YOLO v2

출력: $13 \times 13 \times 125$

- $S = 13$ (그리드 크기)
- $B = 5$ (박스 개수)
- $C = 20$ (클래스)
- 총 845개 예측 박스

Anchor box 도입으로 더 많은 박스를 효율적으로 예측합니다.

YOLO v3

출력: 3개 스케일

- 13×13 (stride 32, 큰 객체)
- 26×26 (stride 16, 중간 객체)
- 52×52 (stride 8, 작은 객체)
- 총 10,647개 예측 박스

Multi-scale로 다양한 크기의 객체를 정밀하게 검출합니다.

주요 특징

- **최초의 실시간 객체 검출 모델:** 45 FPS의 놀라운 속도를 달성하여 실시간 응용의 가능성을 열었음.
- **Map 63.4%:** Faster R-CNN보다 낮지만, 속도는 10배 이상 빠름
- **7×7 그리드 구조:** 각 셀당 2개의 bounding box를 예측하는 간단한 구조
- **단일 네트워크:** 24개 conv layer + 2개 FC layer로 구성된 경량 구조

한계점

- **작은 객체 검출 어려움:** 그리드 해상도가 낮아 작은 객체나 밀집된 객체를 놓치는 경우가 많았음
- **Localization 오류:** 박스 위치 예측 정확도가 Two-Stage 대비 낮았음
- **낮은 Recall:** 객체를 놓치는 경우가 많아 재현율이 낮았음

1

Batch Normalization

모든 Convolutional layer에 Batch Norm을 추가하여 학습 안정성을 높였습니다. +2% mAP 향상 효과를 보였습니다.

2

High Resolution Classifier

입력 이미지 크기를 224×224에서 448×448로 증가시켜 더 세밀한 feature를 추출합니다. +4% mAP 개선되었습니다.

3

Anchor Boxes

Faster R-CNN의 anchor 개념을 도입했습니다. K-means clustering으로 데이터셋에 최적화된 anchor 크기를 자동 결정합니다. Recall 81% → 88%로 크게 향상되었습니다.

4

Dimension Clusters (K-means)

Euclidean 거리 대신 IoU 기반 clustering을 사용합니다: $d(\text{box}, \text{centroid}) = 1 - \text{IoU}(\text{box}, \text{centroid})$. k=5가 복잡도와 성능의 최적 균형점입니다.

5

Fine-Grained Features

26×26 feature map을 13×13으로 재배치하는 Passthrough layer를 추가했습니다. 고해상도 정보를 보존하여 작은 객체 검출을 개선합니다.

6

Multi-Scale Training

10 batch마다 입력 크기를 {320, 352, ..., 608} 범위에서 무작위로 변경합니다. 다양한 해상도에 강건한 모델을 학습시킵니다.

최종 성능: YOLO v2-544는 78.6% mAP, 40 FPS를 달성하여 Fast R-CNN 대비 월등히 빠른 속도를 보였습니다.

아키텍처 특징

- 19개 Convolutional layers + 5개 Max Pooling
- 1×1 Convolution으로 채널 수 감소, 파라미터 효율성 증대
- Global Average Pooling으로 FC layer 제거
- Batch Normalization을 모든 conv layer에 적용

성능 지표

ImageNet에서 74% top-1 accuracy를 달성했습니다. 5.58B operations로 VGG-16 대비 훨씬 효율적이며, 빠른 추론 속도를 제공합니다.

설계 철학

VGG의 간단한 3×3 conv 구조를 따르되, 1×1 conv로 계산량을 줄이는 Network in Network 아이디어를 적용했습니다.

Classification 정확도와 속도의 균형을 맞추어 객체 검출의 backbone으로 최적화되었습니다.

▶ 핵심 혁신: 3개의 스케일 검출

- 총 10,647개 박스를 예측하여 (v2는 845개), 다양한 크기와 종횡비의 객체를 놓치지 않음

82nd Layer

13×13 (stride 32)

큰 객체 검출에 특화. 낮은 해상도의 깊은 feature를 사용하여 의미적 정보가 풍부합니다.

94th Layer

26×26 (stride 16)

중간 크기 객체 검출. Upsampling과 concatenation으로 앞의 layer의 정보를 융합합니다.

106th Layer

52×52 (stride 8)

작은 객체 검출에 최적화. 높은 해상도로 세밀한 위치 정보를 제공합니다.

FPN: Top-down Pathway

YOLO v3는 FPN 구조를 채택하여 다양한 레벨의 feature를 효과적으로 융합합니다.



최종 성능: YOLO v3-608은 33.0% mAP@0.5:0.95, 51ms 추론 시간으로 RetinaNet보다 빠르고 정확합니다.

Darknet-53 Backbone

- 53개 Convolutional layers
- Residual Connections (ResNet 스타일)
- 성능: ResNet-101보다 빠르고 정확
- ImageNet top-1 accuracy 77.2%

Binary Cross-Entropy Loss

Softmax를 제거하고 각 클래스를 독립적으로 예측합니다. Multi-label 지원이 가능해져 "person with dog" 같은 복합 상황을 처리할 수 있습니다.



Input Image

원본 이미지 입력 (예: 640×640×3)



Backbone - Feature Extraction

YOLO v2: Darknet-19

YOLO v3: Darknet-53

YOLO v8: CSPDarknet with C2f modules

다양한 스케일의 feature map 생성



Neck - Feature Aggregation

FPN (Feature Pyramid Network)

PANet (Path Aggregation Network)

다양한 레벨의 feature를 융합하여 풍부한 표현 생성



Head - Detection Head

Classification + Regression

Decoupled Head (분리된 헤드)

Anchor-free (YOLOv8)



Output

Bounding Boxes + Class Probabilities

각 박스: (x, y, w, h, confidence, class scores)

1. Localization Loss (박스 위치)

박스의 중심 좌표와 크기를 예측하는 손실입니다.

$$\lambda_{coord} \times \sum [(x_i - \hat{x}_i)^2 + (y_i - \hat{y}_i)^2 + (\sqrt{w_i} - \sqrt{\hat{w}_i})^2 + (\sqrt{h_i} - \sqrt{\hat{h}_i})^2]$$

✓ 사용 이유: 큰 박스와 작은 박스의 오차를 균형있게 처리합니다. 작은 박스의 작은 오차가 큰 박스의 큰 오차만큼 중요하도록 만듭니다.

2. Confidence Loss (객체 존재)

각 박스에 객체가 있는지 없는지를 예측하는 손실입니다.

$$\sum (C_i - \hat{C}_i)^2 + \lambda_{noobj} \times \sum (C_i - \hat{C}_i)^2$$

두 항의 구분: 객체가 있는 셀과 없는 셀을 별도로 처리합니다. λ_{noobj} (보통 0.5)로 배경 박스의 영향을 줄여 class imbalance 문제를 완화합니다.

3. Classification Loss (클래스)

검출된 객체의 클래스를 예측하는 손실입니다.

$$\sum \sum (p_i(c) - \hat{p}_i(c))^2$$

각 클래스에 대한 확률 분포를 예측합니다. YOLO v3부터는 Binary Cross-Entropy를 사용하여 multi-label 지원이 가능합니다.

Precision과 Recall

$$TP/(TP+FP)$$

Precision (정밀도)

모델이 예측한 것 중 실제로 맞춘 비율

목표: False Positive 최소화

높은 Precision = 예측이 신뢰할 만함

$$TP/(TP+FN)$$

Recall (재현율)

실제 객체 중 모델이 찾아낸 비율

목표: False Negative 최소화

높은 Recall = 객체를 놓치지 않음

Trade-off: Confidence threshold를 낮추면 Recall 증가, Precision 감소. 반대로 threshold를 높이면 Precision 증가, Recall 감소합니다.

mAP (mean Average Precision)

AP (Average Precision): Precision-Recall curve의 면적입니다.

각 클래스별로 계산됩니다.

mAP: 모든 클래스 AP의 평균값입니다.

- mAP@0.5: IoU threshold 0.5에서의 mAP (PASCAL VOC 기준)
- mAP@0.5:0.95: IoU 0.5~0.95 범위의 평균 mAP

(COCO 기준, 더 엄격)

mAP@0.5:0.95는 다양한 IoU에서 성능을 평가하여 더 정확한 localization을 요구합니다.

실무 Threshold 선택 가이드

응용 분야	Threshold	우선순위	이유
보안/의료	0.3 (낮음)	높은 Recall	객체를 놓치면 안 됨
일반 검출	0.5 (중간)	균형	속도와 정확도 절충
광고/추천	0.7 (높음)	높은 Precision	오검출 시 신뢰도 하락

주요 혁신

1

Anchor-Free Detection

Anchor box가 필요 없어 더 유연한 검출이 가능합니다. 다양한 종횡 비와 크기에 자동 적응합니다.

2

C2f Module

CSPNet + Cross Stage Partial 구조로 더 풍부한 gradient flow를 제공합니다. 경량화와 성능을 동시에 달성했습니다.

3

Decoupled Head

Classification과 Regression head를 분리하여 각 task에 최적화했습니다. 성능 향상과 학습 안정성 증대 효과가 있습니다.

4

Task-Aligned Assigner

Positive/negative sample 할당을 개선하여 학습 효율을 높였습니다.

YOLO v8: 최신 기술의 집약 (2023)

모델 라인업 성능

모델	mAP@0.5:0.95	속도
YOLOv8n	37.3%	80+ FPS
YOLOv8s	44.9%	50+ FPS
YOLOv8m	50.2%	30+ FPS
YOLOv8l	52.9%	20+ FPS
YOLOv8x	53.9%	15+ FPS

모델 선택 가이드



엣지 디바이스 / 실시간 필수

추천: YOLOv8n/s

경량 모델로 모바일, IoT 기기에서 실시간 처리 가능. 제한된 연산 자원에서 최적의 성능을 발휘합니다.



고정확도 필요 / 서버 배포

추천: YOLOv8l/x

최고 수준의 mAP를 제공하여 정밀 검출이 필요한 응용에 적합. 충분한 GPU 자원이 있는 서버 환경에서 사용됩니다.



균형잡힌 성능 / 범용

추천: YOLOv8m

속도와 정확도의 최적 균형. 대부분의 실무 상황에서 좋은 선택입니다. Multi-scale 검출이 우수합니다.

Object Detection & Segmentation

YOLOv8을 활용한 Object Detection

1. YOLOv8을 활용한 Object Detection

강의 목표와 학습 경로

01

객체 검출 원리 이해

YOLO의 핵심 작동 방식과 One-Stage

Detector의 강점을 파악하여 실시간 객체 검출의 기초를 다집니다.

02

YOLOv8 실습 숙달

최신 Ultralytics YOLOv8 프레임워크를

사용하여 실제 데이터로 모델을 학습하고 성능을 평가합니다.

03

모델 최적화 및 배포

하이퍼파라미터 튜닝과 ONNX 변환을 통해 실전 배포 가능한 수준의 모델을 완성합니다.

▶ 객체 검출이란

- 이미지 분류
 - 이미지 전체가 '무엇'인지 판별합니다. 예를 들어 "이 사진은 고양이입니다"라고 답합니다.
- 객체 검출
 - 이미지 속 모든 객체의 '위치'와 '종류'를 찾아 표시합니다. "50% 확률로 이 위치에 사람, 92% 확률로 이 위치에 버스"처럼 정확한 좌표와 신뢰도를 제공합니다.

▶ One-Stage Detector

Two-Stage Detector

예시: Faster R-CNN



1단계: 후보 영역 제안

Region Proposal Network가 객체가 있을 만한 영역을 먼저 제안합니다.



2단계: 분류 및 정제

제안된 영역을 분류하고 박스를 정확하게 조정합니다.

높은 정확도, 느린 속도

One-Stage Detector

예시: YOLO (You Only Look Once)

단일 단계 예측

이미지를 한 번만 보고 위치와 분류를 동시에 예측합니다.

실시간 처리 가능, 우수한 정확도

YOLOv8의 주요 특징



Anchor-Free 설계

미리 정의된 앵커 박스 없이 객체의 중심점과 크기를 직접 예측하여 모델이 더 유연하고 단순해졌습니다.



통합 프레임워크

객체 검출, Segmentation(영역 분할), Classification(분류), Pose Estimation(자세 추정)을 하나의 프레임워크로 처리 가능합니다.



다양한 모델 크기

n(nano)부터 x(xlarge)까지 제공됩니다. 작은 모델(n, 3.2M 파라미터)은 빠르고, 큰 모델(x, 68.2M 파라미터)은 정확도가 높습니다.

Object Detection & Segmentation

2. 다중 스케일 구조로 정확도
높이기

2. 다중 스케일 구조로 정확도 높이기

▶ FPN과 PAN의 원리

- 핵심 문제: 깊은 신경망은 추상적인 의미(Semantic) 정보는 강하지만, 작은 객체의 위치(Spatial) 정보는 손실됩니다.

Feature Pyramid Network (FPN)
상위(깊은) 계층의 의미 정보를 하위(얕은) 계층으로 전달(Top-down Pathway)하여, 저수준 피쳐 맵에서도 객체의 종류를 정확하게 파악하도록 돕습니다.

1

2

Path Aggregation Network (PAN)

FPN에 역방향(Bottom-up Pathway) 경로를 추가하여, 하위 계층의 정교한 위치 정보를 상위 계층에 다시 전달합니다. 양방향 융합으로 성능을 극대화합니다.

YOLOv8 아키텍처의 다중 스케일

P3 (80×80)

작은 객체 검출

8×8 ~ 32×32 픽셀 크기의 객체를 담당합니다.

P4 (40×40)

중간 객체 검출

32×32 ~ 96×96 픽셀 크기의 객체를 담당합니다.

P5 (20×20)

큰 객체 검출

96×96+ 픽셀 크기의 객체를 담당합니다.

Backbone(CSPDarknet)에서 특징을 추출한 후, Neck(PAN-FPN)을 통해 다양한 크기의 피쳐 맵(Feature Map)을 생성하여 모든 크기의 객체를 효과적으로 검출합니다.

Object Detection & Segmentation

3. Loss Function

▶ Loss Function의 이해

모델이 학습할 때 '얼마나 틀렸는지'를 측정하는 손실 함수(Loss Function)는 객체 검출 성능의 핵심입니다. YOLOv8은 두 가지 주요 손실을

Box Loss: 위치 오차

계산: 예측 박스(pred)와 정답 박스(gt)의 위치 오차를 계산합니다.

$$\text{Loss} = 1 - \text{CIoU}$$

CIoU (Complete IoU): 단순히 겹치는 영역(IoU)뿐만 아니라, **중심점 거리**와 **종횡비**까지 고려하는 가장 정교한 손실 함수입니다. YOLOv8의 기본 Box Loss로 사용됩니다. 예를 들어, 두 박스가 절반만 겹치더라도 중심이 가까우면 더 나은 예측으로 평가합니다.

Classification Loss: 클래스 오차

계산: 객체의 종류(클래스)를 얼마나 정확하게 예측했는지 계산합니다.

$$\text{FL} = -\alpha (1 - p)^{\gamma} \log(p)$$

Focal Loss (FL): 배경처럼 **쉬운 샘플**의 손실 기여도를 줄이고(γ 파라미터 사용), **어려운 샘플**에 집중하여 학습함으로써 클래스 불균형 문제를 해결합니다. 이미지 속 대부분은 배경이고 객체는 소수이므로, 어려운 객체 검출에 더 많은 학습 노력을 집중시킵니다.

☐ Loss가 낮을수록 모델의 예측이 정답에 가까워집니다. 학습 과정에서 이 Loss 값들이 점점 감소하는 것을 확인할 수 있습니다.

Object Detection & Segmentation

4. 실전 데이터셋 구축하기

▶ Safety Helmet 데이터셋 구조



데이터셋 다운로드

Kaggle에서 Safety Helmet 데이터셋을 다운로드합니다. 총 5000개의 이미지가 포함되어 있습니다.



클래스 확인

데이터셋에는 'head', 'helmet', 'person' 총 3개의 클래스가 포함되어 있음을 확인합니다.



어노테이션 변환

원본 데이터의 XML(.xml) 형식 라벨을 YOLOv8이 사용하는 .txt 형식으로 변환합니다. 각 파일에는 객체의 클래스와 좌표 정보가 포함됩니다.



데이터 분할

전체 5000개 이미지를 Train(3693개), Validation(1175개), Test(537개)로 분할하여 YOLOv8 학습 형식에 맞춥니다.

Train 데이터로 모델을 학습하고, Validation 데이터로 성능을 모니터링하며, Test 데이터로 최종 성능을 평가합니다.

4. 실전 데이터셋 구축하기

▶ PyTorch 데이터 파이프라인



transforms: 이미지 전처리

Resize

224 × 224 픽셀로 크기 통일

ResNet18 입력 요구사항

ToTensor

이미지를 PyTorch 텐서로 변환

픽셀 값 범위: 0~1로 정규화

Normalize

평균과 표준편차로 표준화

ImageNet 통계 값 사용

☐ 편의점 알바 비유: Dataset은 진열대에서 물건을 하나씩 꺼내는 것이고, DataLoader는 그 물건들을 카트에 32개씩 담아서 한 번에 계산대로 가져오는 것입니다. 배치 크기(batch size)가 바로 카트의 크기입니다.

Object Detection & Segmentation

5. 학습 성능 향상 기법

▶ Data Augmentation

- 데이터 증강(Data Augmentation)은 제한된 데이터셋으로도 모델의 일반화 성능을 크게 향상시키는 핵심 기법입니다.
- YOLOv8은 다양한 증강 기법을 자동으로 적용합니다.

Mosaic

Augmentation

4장의 이미지를 모자이크처럼 이어 붙여 새로운 이미지를 생성합니다. 다양한 크기(Scale)의 객체를 한 번에 학습하여 일반화 성능을 높입니다.

Mixup

두 이미지를 섞어서(Blend) 새로운 이미지를 만드는 기법입니다. 모델이 더 부드러운 결정 경계를 학습하도록 돕습니다.

Copy-Paste

이미지 내 객체를 복사-붙여넣기하여 Hard Sample(어려운 샘플)을 인위적으로 생성합니다. 특히 작은 객체나 가려진 객체 검출 능력을 향상시킵니다.

▶ Strong Augmentation 파라미터

학습률(lr)과 모멘텀(momentum) 외에도 다양한 파라미터로 증강 강도를 조절할 수 있습니다.

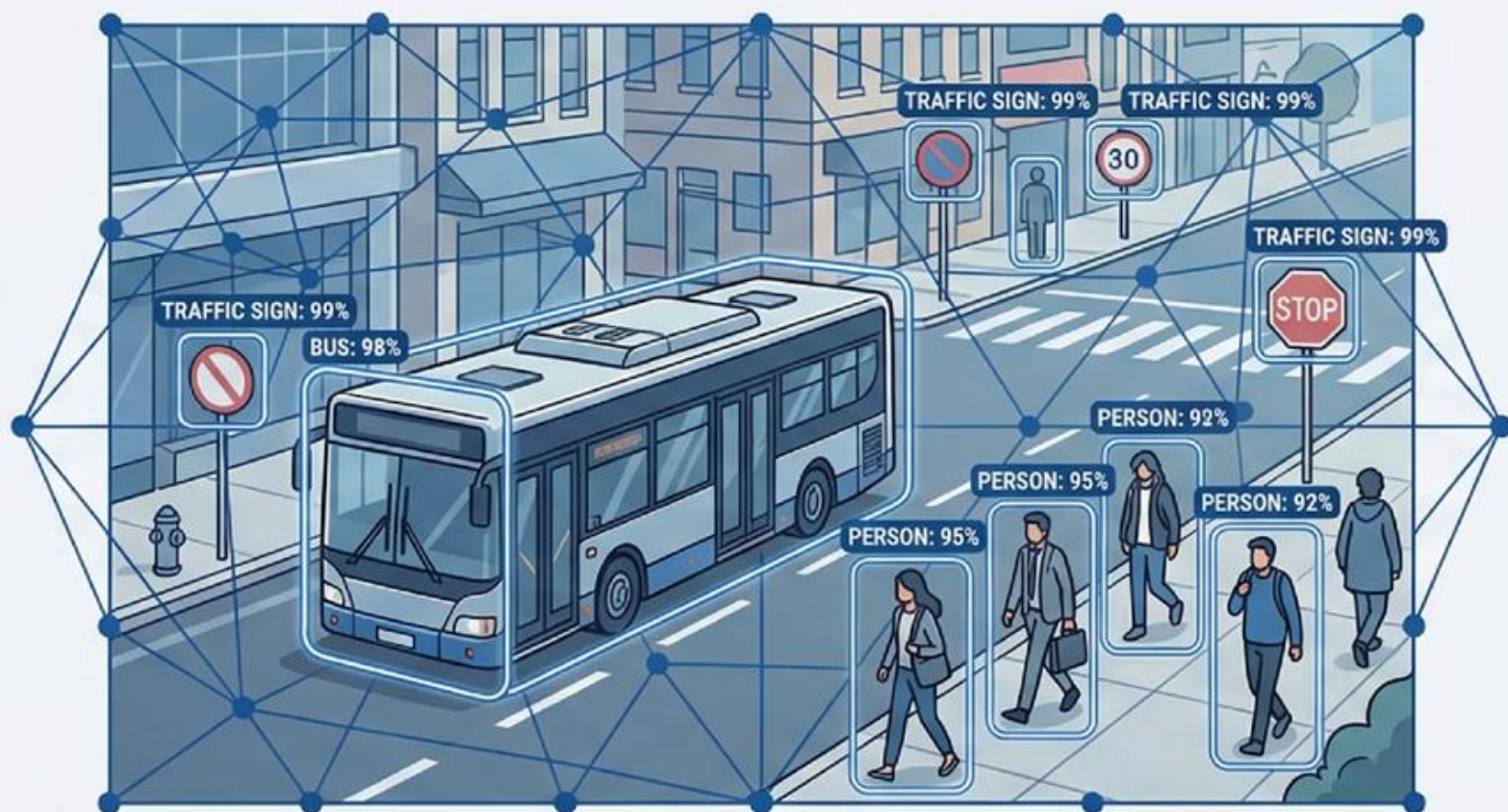
hsv_h, hsv_s, hsv_v: 색상(Hue), 채도(Saturation), 명도(Value) 변형

translate: 이미지 이동 변환 scale: 크기 조절 degrees: 회전 각도

이러한 기하학적 및 색상 변형을 강하게 적용하여 모델의 강인함(Robustness)을 확보합니다.

실전 객체 검출 마스터하기: YOLOv8 완벽 가이드

최신 AI가 이미지 속 모든 객체의 위치와 종류를 동시에 찾아내는 핵심 기술을 배워봅니다



YOLOv8을 활용한 실제 데이터 학습부터 실무 배포까지 완전 정복

01

Chapter 1: 객체 검출의 세계로

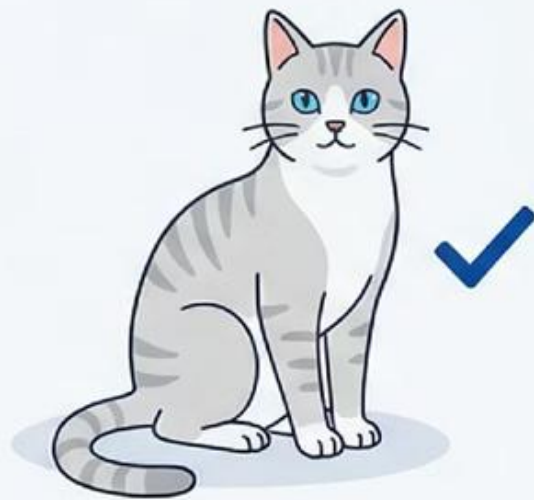
...



‘AI가 세상을 보는 두 가지 방법’

분류 (Classification)

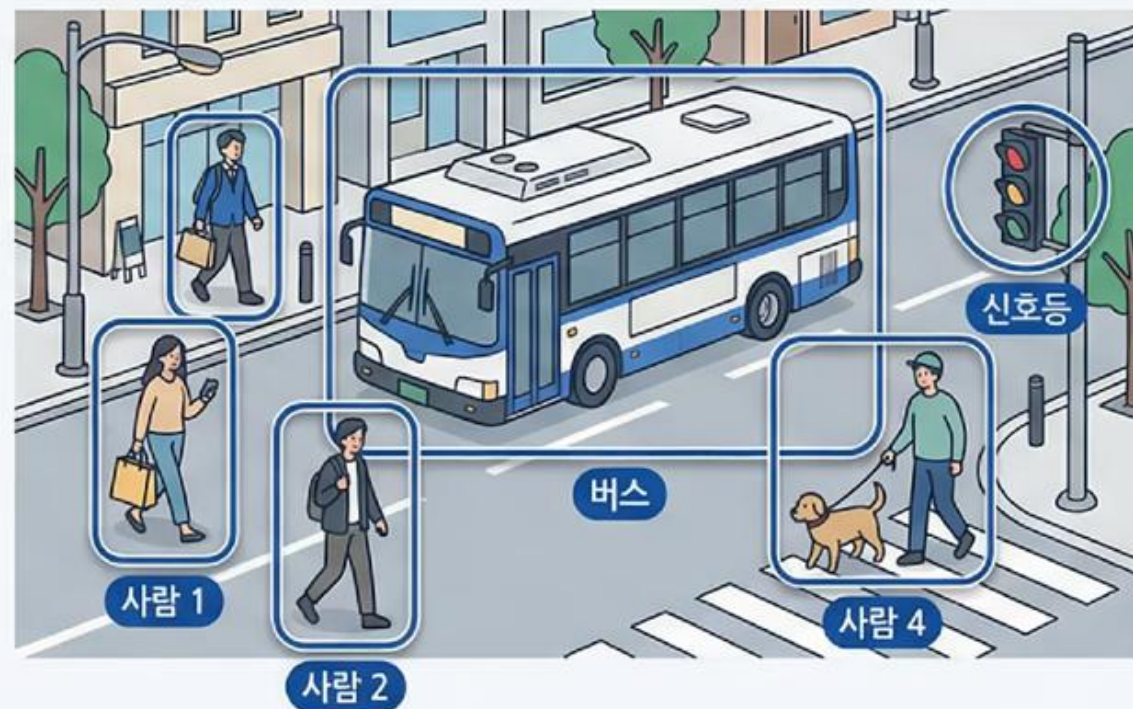
분류는 ‘무엇’인지만 답하지만,
검출은 ‘무엇’이 ‘어디에’ 있는지를 모두 찾아냅니다.



고양이

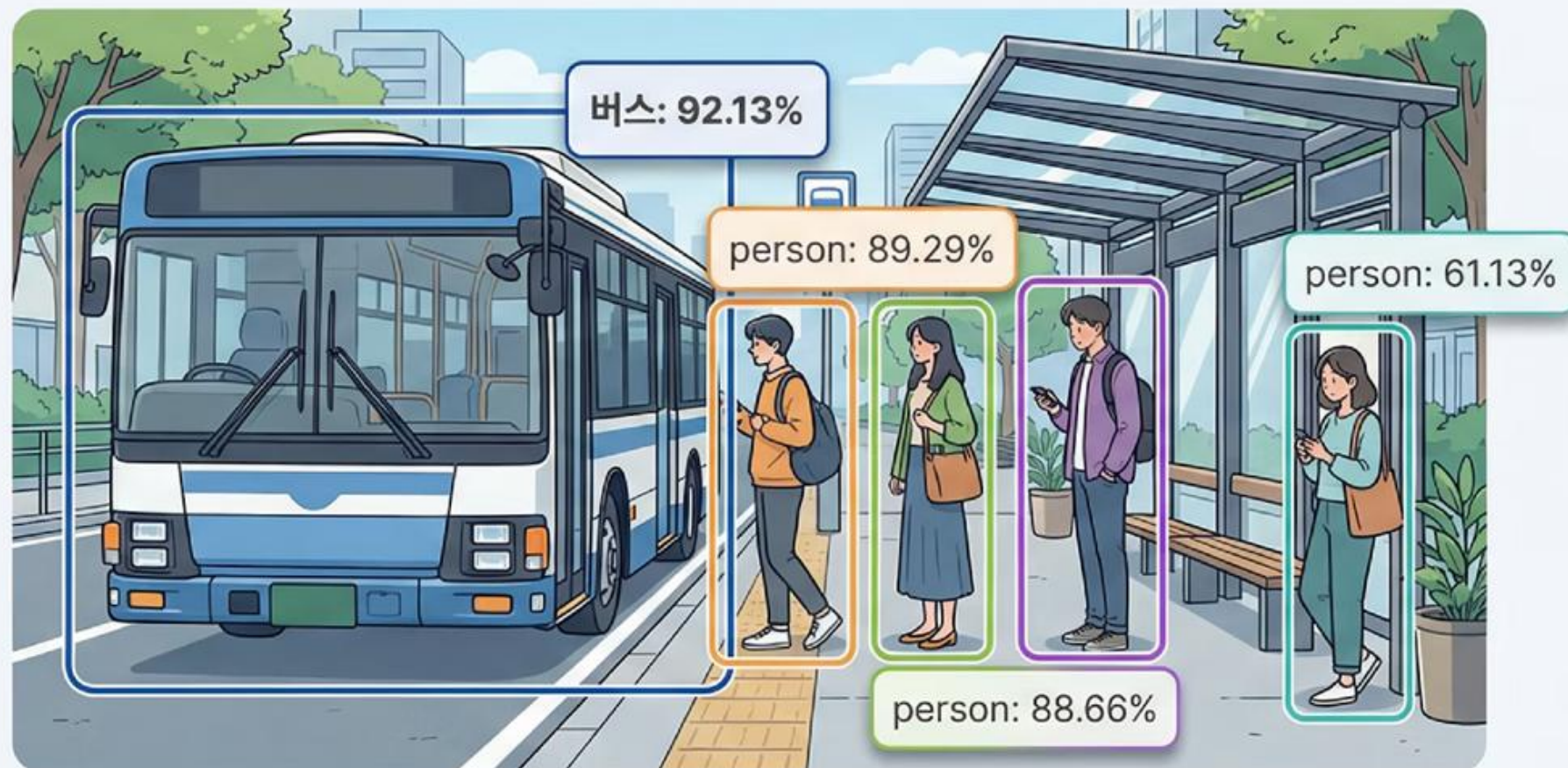
검출 (Detection)

하나의 이미지에서 버스 1개와 사람 4명을
동시에 발견하고 각각의 정확한 위치를 표시합니다.



92.13% 확률로 발견된 버스

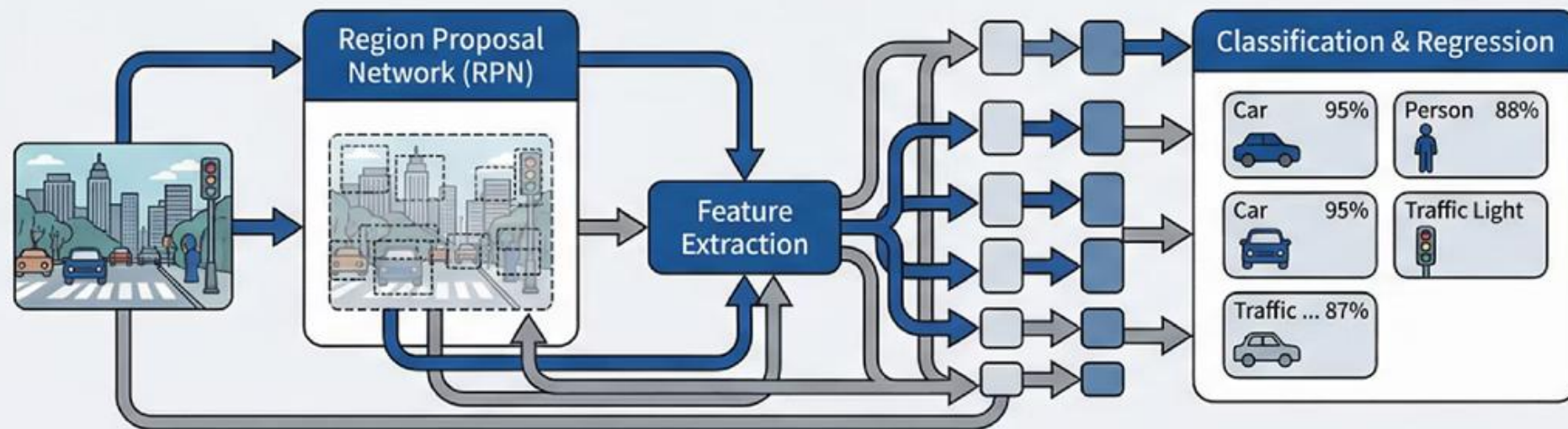
실제 YOLOv8 추론 결과를 보여주는 화면 - 버스 정거장 장면에서 색색의 바운딩 박스와 신뢰도 수치들이 표시된 생생한 검출 결과



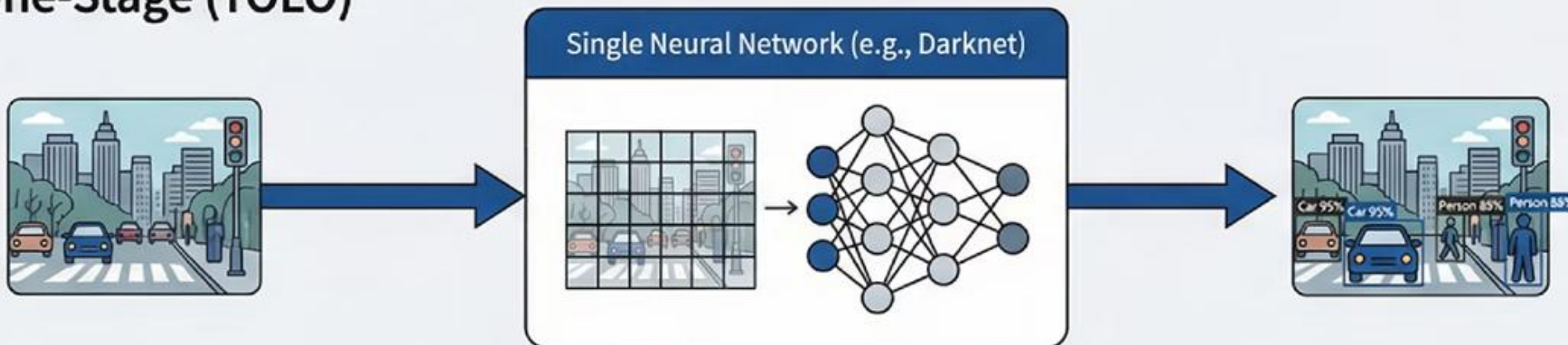
'YOLO의 혁신: 한 번에 모든 것을 본다'

Two-Stage (기존 방식)

기존 방식은 2단계로 나누어 처리했지만, YOLO는 단 한 번만 보고 모든 객체를 찾아냅니다



One-Stage (YOLO)



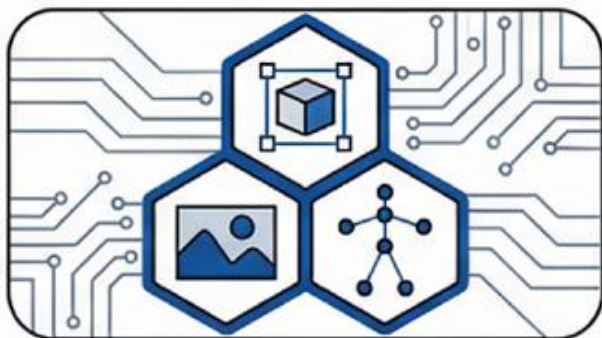
실시간 처리가 가능한 이유는 바로 이 One-Stage 방식 때문입니다

YOLOv8의 3가지 핵심 혁신



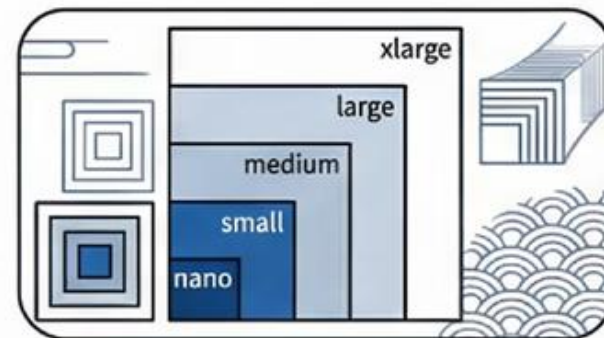
1. Anchor-Free 설계

앵커 박스 없이도 정확한 예측이 가능한
Anchor-Free 설계로 모델이
더욱 유연해졌습니다



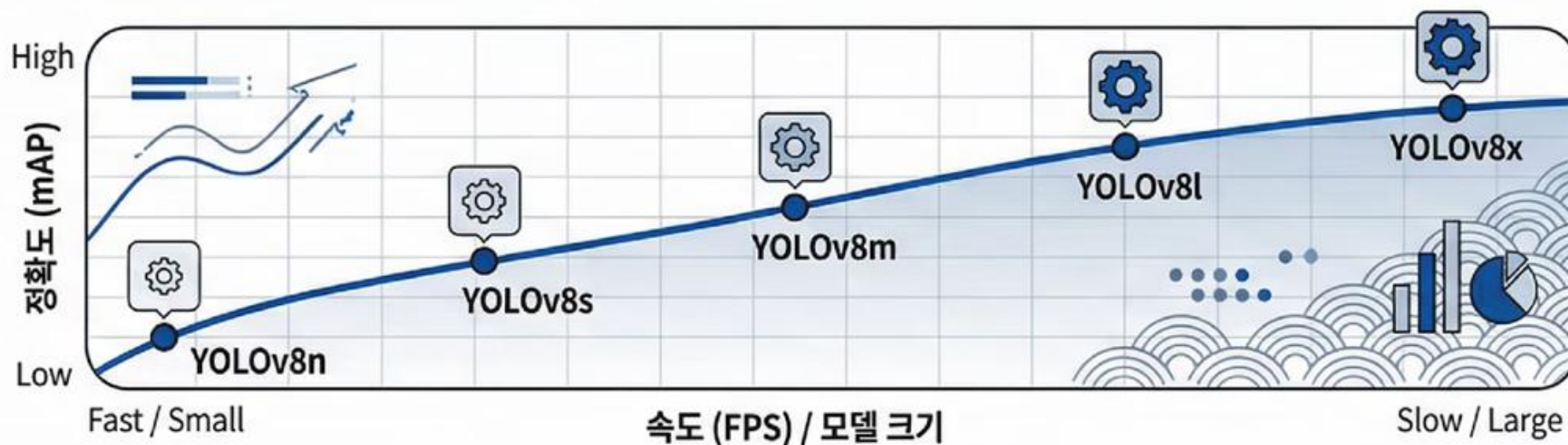
2. 통합 프레임워크

객체 검출, 영역 분할, 자세 추정을
하나의 통합 프레임워크에서 처리합니다



3. 다양한 모델 크기

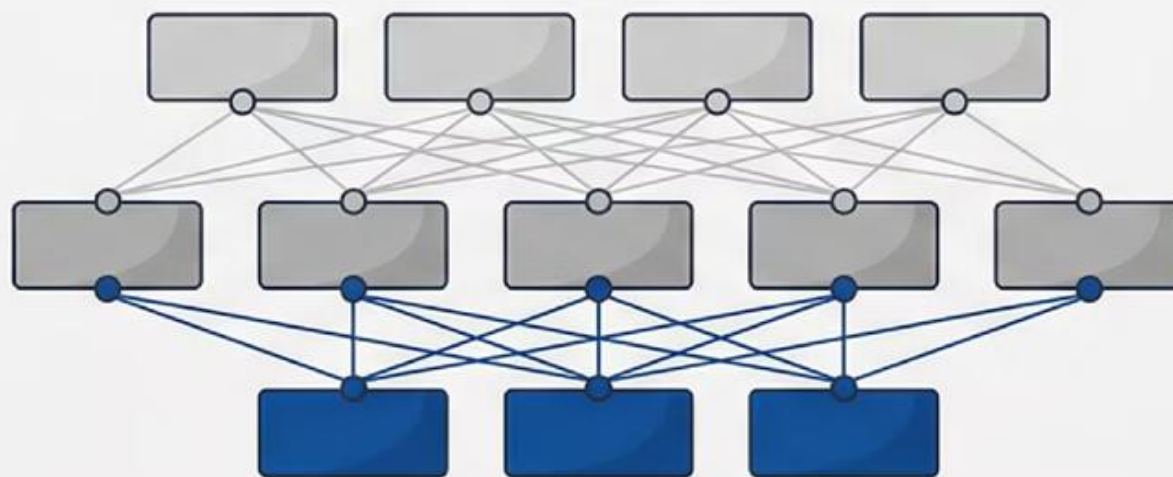
nano부터 xlarge까지, 상황에 맞는
최적의 모델 크기를 선택할 수 있습니다



02

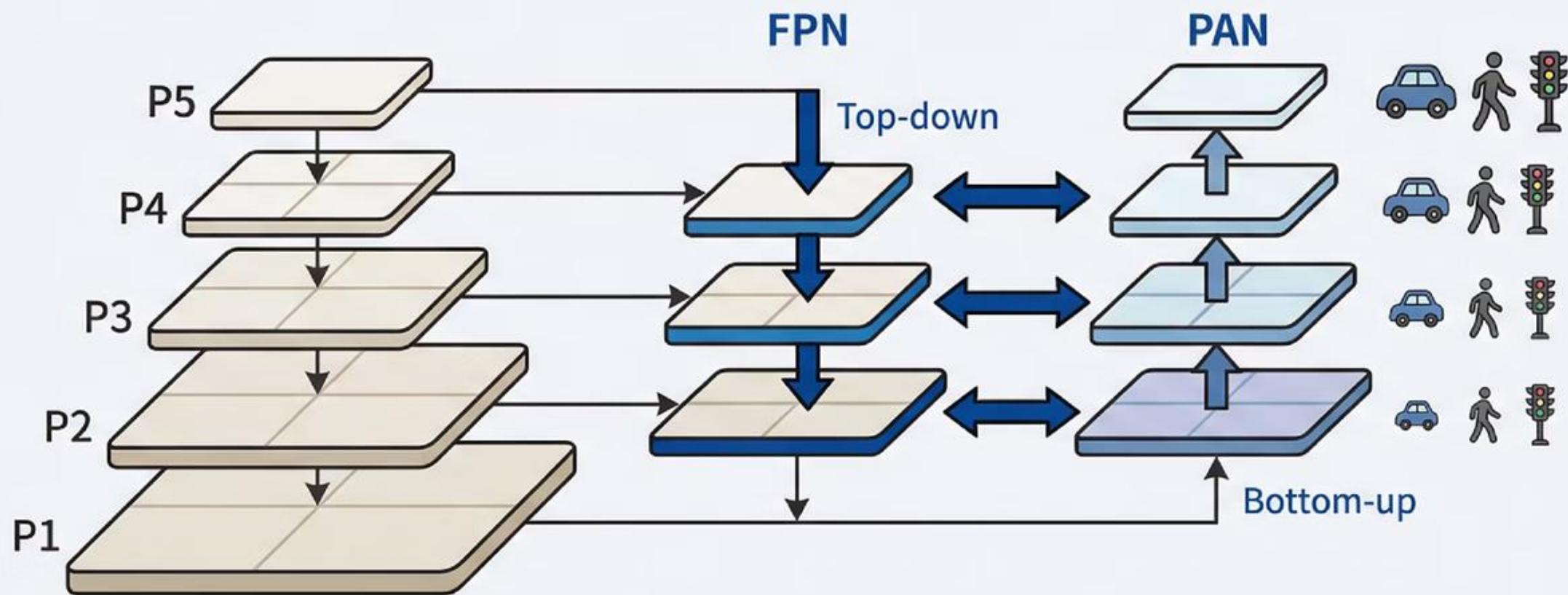
Chapter 2: 아키텍처의 비밀

...



작은 객체를 놓치지 않는 비밀: 다중 스케일

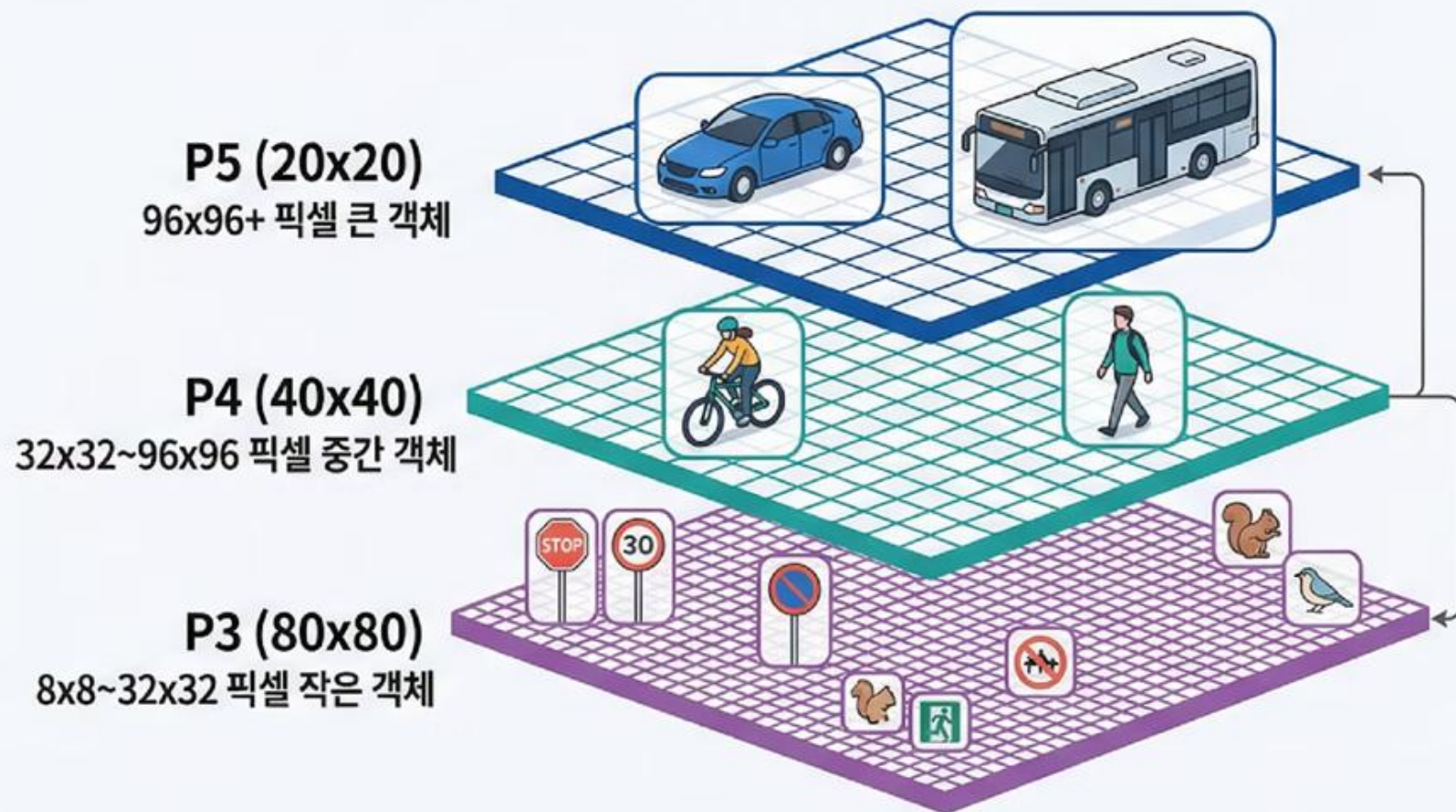
깊은 신경망은 의미는 잘 파악하지만 작은 객체의 위치 정보를 잃어버리는 문제가 있습니다



FPN은 상위 계층의 의미 정보를 하위로, PAN은 하위 계층의 위치 정보를 상위로 전달합니다

P3, P4 P3, P4, P5: 크기별 전문가들

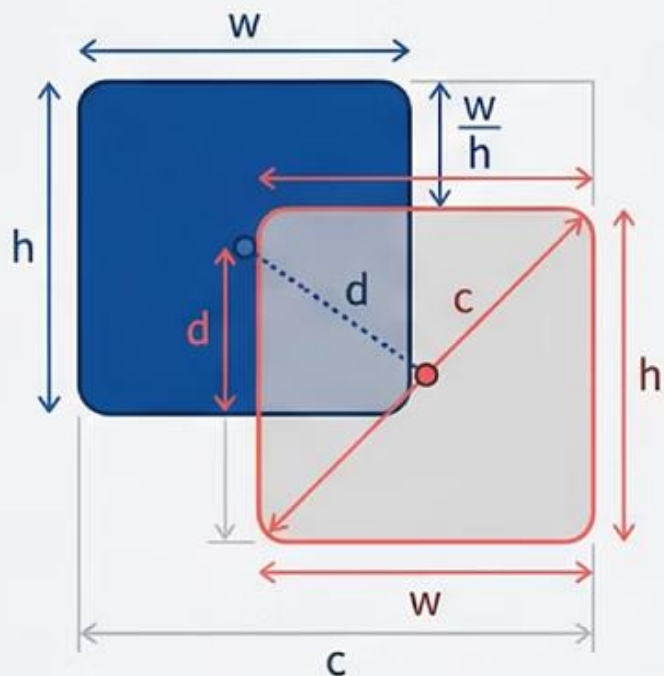
P3는 8x8~32x32 픽셀의 작은 객체를, P5는 96x96+ 픽셀의 큰 객체를 담당합니다



각기 다른 해상도의 피쳐 맵이 서로 다른 크기의 객체들을 전문적으로 검출합니다

정확도를 결정하는 수학적 원리

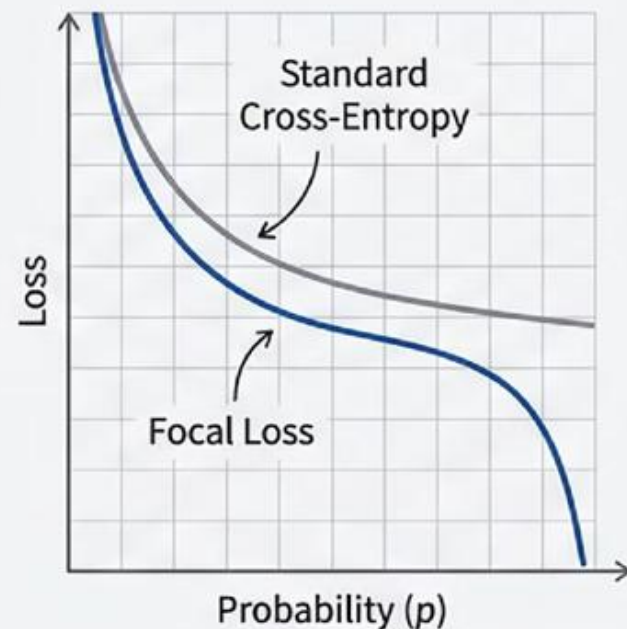
CloU Loss (Box Loss)



$$Loss = 1 - \text{CloU}$$

CloU Loss는 단순한 겹침뿐만 아니라
중심점 거리와 종횡비까지 모두 고려합니다

Focal Loss (Classification Loss)



$$FL = -\alpha(1-p)^\gamma \log(p)$$

Focal Loss는 쉬운 배경 샘플의 기여도를 줄이고
어려운 객체에 집중하여 학습합니다

03

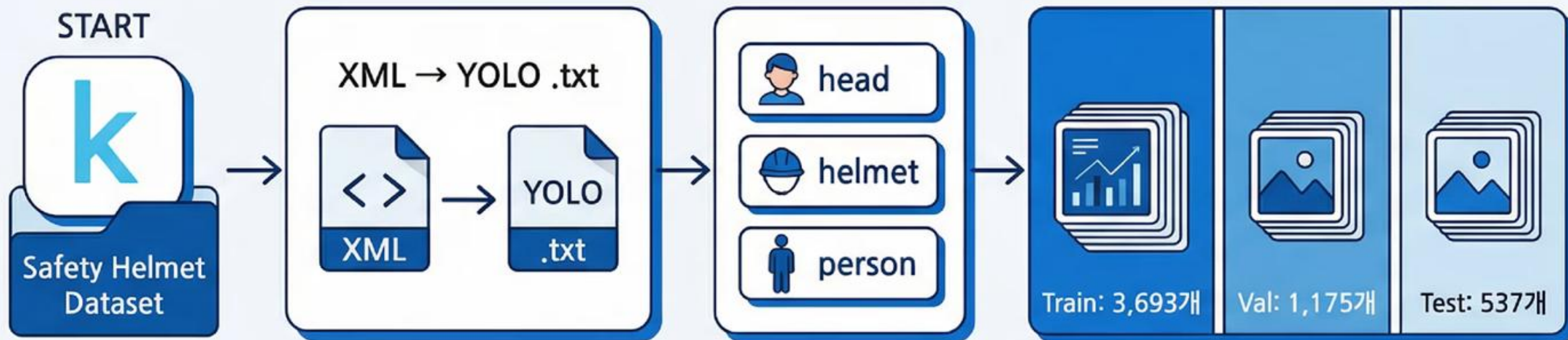
Chapter 3: 실전 학습의 기술

...



5000장의 안전모 데이터셋 준비

Kaggle의 Safety Helmet 데이터셋을 XML에서 YOLO 형식으로 변환합니다



Train 3693개, Validation 1175개, Test 537개로 과학적으로 분할합니다.
head, helmet, person 3개 클래스로 구성된 데이터셋과 변환 과정을 보여주는 화면

모자이크의 마법: 4장이 1장으로

Mosaic Augmentation은 4장의 이미지를
퍼즐처럼 맞춰 새로운 학습 이미지를 생성합니다

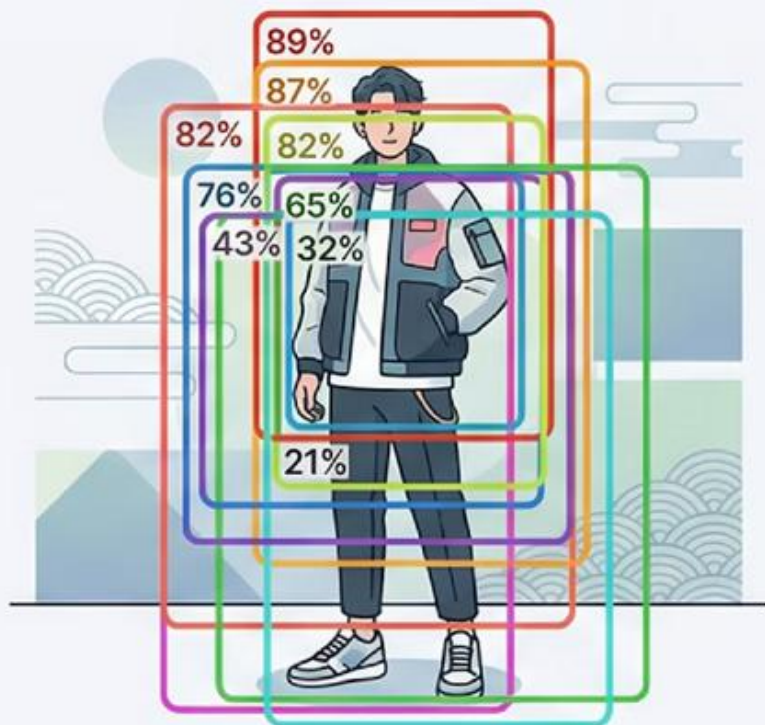


다양한 크기의 객체들을 한 번에 학습하여
모델의 일반화 성능을 극대화합니다

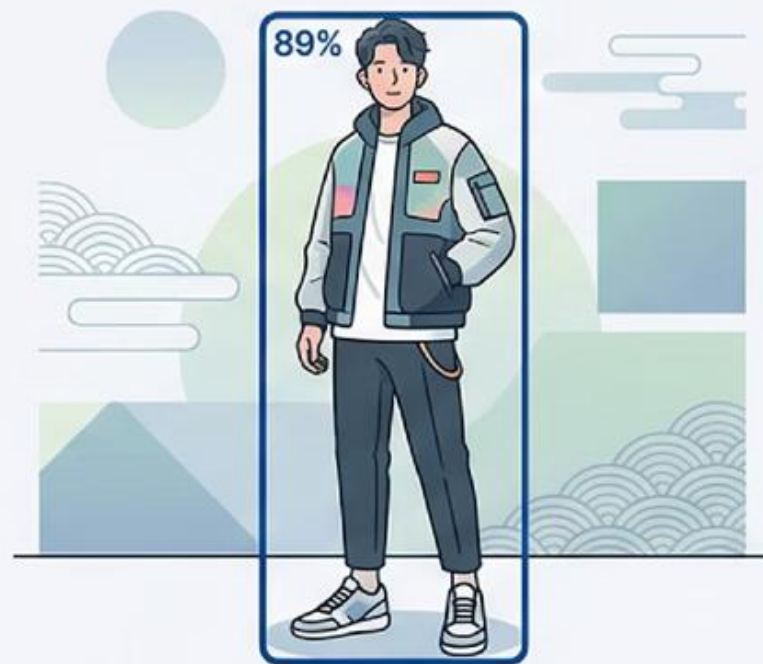
NMS: 중복 제거의 핵심 알고리즘

하나의 객체에 여러 개의 박스가 겹칠 때, 가장 확실한 하나만 남기고 나머지를 제거합니다

Before NMS



After NMS



conf_thres



iou_thres



Confidence Threshold는 정밀도를,
IoU Threshold는 중복 제거 정도를 조절합니다

실무 배포: 속도 최적화의 완성

PyTorch 모델을 ONNX로 변환하고 TensorRT로 최적화하여 추론 속도를 극대화합니다



실시간 처리가 필요한 실무 환경에서는 속도 최적화가 성공의 핵심입니다.
모델 변환 과정과 함께 속도 비교 차트, 실시간 객체 검출이 동작하는 모니터 화면